

04: Kubernetes Operators and Kubebuilder, Deep Dive

An operator extends Kubernetes with a new resource type and a controller that manages it. To build one, or to answer interview questions about one, you need to understand four layers: the Kubernetes API conventions a custom resource follows, the controller-runtime library that watches resources and schedules work, the code Kubebuilder generates, and the reconciliation logic you write.

This chapter covers those layers in that order. The examples refer to the GpuWorkload operator in this repository's operator/ directory.

This chapter covers

- What an operator is, and how it differs from a Helm chart
- API groups, versions, kinds, CRDs, structural schemas, and the status subresource
- How controller-runtime turns watch events into calls to Reconcile
- Writing a reconciler: fetching objects, CreateOrUpdate, owner references, finalizers, and status conditions
- The Kubebuilder project layout, markers, and code generation
- Admission webhooks, leader election, metrics, and health probes
- A design checklist and common interview questions

Contents

1. What is an operator?	3
2. Kubernetes API conventions	4
2.1 Group, version, kind, and resource	4
2.2 CRDs and custom resources	4
2.3 Structural schemas	4
2.4 The status subresource	5
2.5 Versions and conversion	5
3. How controller-runtime works	6
3.1 From watch events to Reconcile	6
3.2 The manager, cache, and clients	7
3.3 Registering watches with the builder	7
3.4 The Reconcile contract	8
3.5 Level-triggered and edge-triggered logic	8
3.6 Rate limiting and backoff	9
4. Writing a reconciler	10
4.1 The structure of Reconcile	10
4.2 CreateOrUpdate	11

4.3 Objects from the cache	11
4.4 Owner references and garbage collection.....	11
4.5 Finalizers	12
4.6 Status, conditions, and observedGeneration.....	13
4.7 Update conflicts.....	14
5. The Kubebuilder project.....	16
5.1 Layout	16
5.2 The PROJECT file	16
5.3 Markers	16
5.4 Code generation	17
5.5 RBAC markers	17
5.6 The manager entry point	18
5.7 Kubebuilder commands	18
6. Admission webhooks	19
6.1 The admission sequence	19
6.2 Defaulting webhooks	19
6.3 Validating webhooks	19
7. Leader election, metrics, and health.....	20
7.1 Leader election	20
7.2 Metrics.....	20
7.3 Health and readiness	20
8. Testing an operator	21
9. Operator design checklist	22
10. The GPU Operator as a case study.....	23
11. Interview questions	24
What is the difference between a controller and an operator?	24
Why are controllers level-triggered?	24
Why must nvidia.com/gpu be set in limits?	24
What happens to a workload if the operator stops?	24
How do you upgrade an operator safely?	24
What metrics would you monitor for an operator?.....	24
Summary	25

1. What is an operator?

An operator is a **controller for a custom resource** that encodes operational knowledge in code. Kubernetes already does this for its built-in resources: the Deployment controller knows how to roll out a new version, and the ReplicaSet controller knows how to keep a number of Pods running. An operator applies the same approach to your own domain objects. It consists of three parts:

- A **CustomResourceDefinition (CRD)** that adds a typed resource to the Kubernetes API.
- A **controller** that watches that resource, and usually the resources it creates.
- **Reconciliation logic** that creates, updates, or deletes resources until the actual state matches the desired state in the resource's spec.

Well-known operators include:

Operator	What it manages
NVIDIA GPU Operator	Node-level GPU software, driven by a ClusterPolicy resource: the driver, container toolkit, device plugin, DCGM exporter, MIG manager, and validators
Kubeflow Training Operator	Distributed training jobs such as PyTorchJob
Prometheus Operator	Prometheus and Alertmanager deployments and their configuration

Operators compared with Helm charts. A Helm chart renders templates and applies them once, when you install or upgrade. An operator runs continuously: it detects and repairs drift, reacts to failures, performs upgrades step by step, and cleans up external resources on deletion.

2. Kubernetes API conventions

2.1 Group, version, kind, and resource

Every Kubernetes API type is identified by four names:

Name	Meaning	Examples
Group	A family of related APIs	apps, batch, gpucloud.example.com (the core group has an empty name)
Version	The API version within the group	v1, v1beta1
Kind	The type name used in YAML and Go	Deployment, GpuWorkload
Resource	The lowercase plural used in URLs	deployments, gpuworkloads

The REST path for a namespaced resource is `/apis/<group>/<version>/namespaces/<namespace>/<resource>`, for example `/apis/gpucloud.example.com/v1/namespaces/default/gpuworkloads/triton-demo`. Run `kubectl api-resources` to see the kinds, resources, and short names that a cluster serves.

2.2 CRDs and custom resources

- A **CustomResourceDefinition** is itself a Kubernetes object. It registers a new resource type with the API server and defines its schema.
- A **custom resource** is an instance of that type, for example a `GpuWorkload` named `triton-demo`.

`kubectl apply -f config/crd/bases/` installs the CRD. Applying a file from `config/samples/` creates a custom resource.

2.3 Structural schemas

A CRD in `apiextensions.k8s.io/v1` must include a *structural* OpenAPI v3 schema, which specifies a type for every field. The API server uses the schema to:

- Validate objects when they are created or updated.
- Remove fields that the schema does not define (*pruning*), unless a field is marked with `x-kubernetes-preserve-unknown-fields`.
- Apply default values.
- Publish the schema, which `kubectl explain` and client generators use.

In a Kubebuilder project, `controller-gen` generates the schema from the Go types and their markers. Do not edit `config/crd/bases/gpucloud.example.com_gpuworkloads.yaml` by hand; change the Go types and regenerate it.

2.4 The status subresource

The marker `+kubebuilder:subresource:status` enables the `/status` subresource. It separates the object into two parts that are written through different endpoints:

- An update to the main resource ignores changes to status.
- An update to `/status` ignores changes to everything except status.
- `metadata.generation` increases only when the spec changes, not when the status changes.

The separation has practical benefits. Users and controllers cannot overwrite each other's data by accident, RBAC can grant a controller permission to write status without granting permission to change the spec, and generation becomes a reliable counter of spec changes for `observedGeneration` (section 4.6).

2.5 Versions and conversion

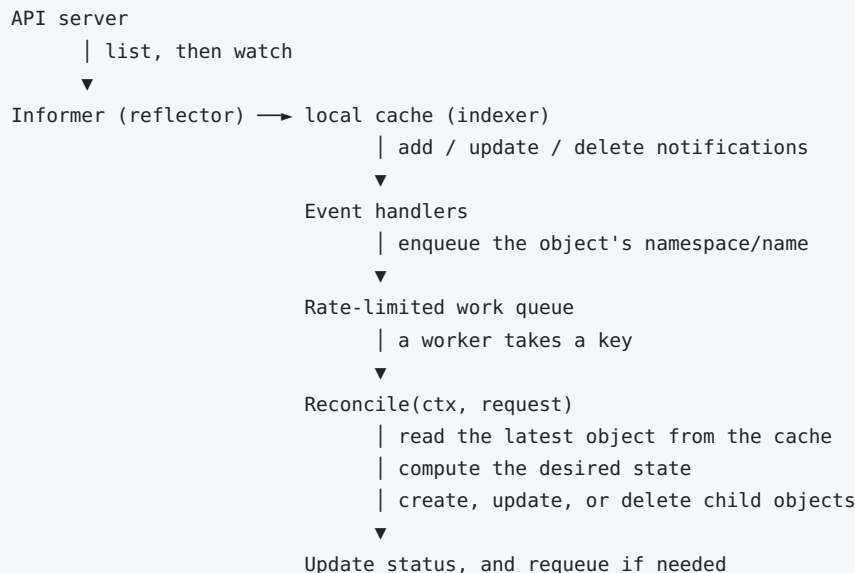
A CRD can serve several versions at once, such as `v1beta1` and `v1`. Exactly one version is the **storage version**, the form in which objects are saved in etcd. When a client reads or writes a different version, the API server converts the object.

- If the versions have the same schema, the API server converts them by changing only `apiVersion` (the `None` conversion strategy).
- If the schemas differ, you provide a **conversion webhook**. In Kubebuilder, you mark one version as the *hub* and implement conversion to and from the hub for each other version.

3. How controller-runtime works

3.1 From watch events to Reconcile

A controller-runtime controller is **level-triggered**: it does not act on individual events. Events only tell it which objects to look at. The path from the API server to your code is:



Four properties of this design are important:

1. **The informer keeps a local cache.** It lists every object of a type once, then receives changes through a watch. Reads in Reconcile come from this cache, so they do not load the API server.
2. **The work queue deduplicates keys.** If ten events for the same object arrive before a worker handles it, the queue holds a single entry, and Reconcile runs once.
3. **A key is never processed by two workers at the same time,** even when the controller has several workers.
4. Reconcile **receives only a namespace and name,** not the event or the object. It must read the object's current state.

3.2 The manager, cache, and clients

Component	Role
Manager	Owns the shared dependencies: the scheme, cache, clients, leader election, the metrics and health servers, the webhook server, and the controllers. <code>mgr.Start</code> runs them all
Cache	Informer-backed storage for reads. Fast, but can lag slightly behind the API server
Client (<code>mgr.GetClient()</code>)	Reads <code>Get</code> and <code>List</code> from the cache, and sends writes directly to the API server
API reader (<code>mgr.GetAPIReader()</code>)	Reads directly from the API server, for the rare case that needs the latest value

Because the cache can lag, a `Get` immediately after a `Create` may not find the new object. Controllers handle this by being idempotent: the next reconcile sees the object.

3.3 Registering watches with the builder

The builder declares which objects the controller watches and how their events map to reconcile requests. The `GpuWorkload` controller in this repository uses:

```
func (r *GpuWorkloadReconciler) SetupWithManager(mgr ctrl.Manager) error {
    return ctrl.NewControllerManagedBy(mgr).
        For(&gpucloudv1.GpuWorkload{}).
        Owns(&appsv1.Deployment{}).
        Owns(&corev1.Service{}).
        Complete(r)
}
```

Method	Effect
<code>For(&GpuWorkload{})</code>	Reconcile a <code>GpuWorkload</code> whenever it changes
<code>Owns(&Deployment{})</code>	When a <code>Deployment</code> changes, reconcile the <code>GpuWorkload</code> listed as its controller owner. <code>Deployment</code> status changes reach the reconciler this way
<code>Watches(&corev1.Node{}, handler.EnqueueRequestsFromMapFunc(fn))</code>	Map any object to one or more reconcile requests, for example to react when a node's GPU labels change
<code>WithEventFilter(p)</code>	Apply a predicate to every watch
<code>builder.WithPredicates(p)</code> passed to <code>For</code> , <code>Owns</code> , or <code>Watches</code>	Apply a predicate to one watch

Warning: `WithEventFilter(predicate.GenerationChangedPredicate{})` filters all watches, including `Owns`. A `Deployment`'s status changes do not change its generation, so the filter would stop the controller from seeing replicas become ready, and the custom resource's status would never update. To ignore status-only changes to the primary resource, attach the predicate to `For` only:

```
go For(&gpucloudv1.GpuWorkload{},
builder.WithPredicates(predicate.GenerationChangedPredicate{}))
```

3.4 The Reconcile contract

Every reconciler implements one method:

```
func (r *GpuWorkloadReconciler) Reconcile(
    ctx context.Context,
    req ctrl.Request,
) (ctrl.Result, error)
```

The return value tells the controller what to do next:

Return	Meaning
<code>ctrl.Result{}, nil</code>	Success. Do not requeue until another event arrives
<code>ctrl.Result{}, err</code>	Failure. The controller logs the error, increments the error metric, and requeues with exponential backoff
<code>ctrl.Result{RequeueAfter: 30 * time.Second}, nil</code>	Success, but reconcile again after the delay. Use this to poll external systems that do not send events
<code>ctrl.Result{Requeue: true}, nil</code>	Requeue with the rate limiter's backoff. Recent controller-runtime releases deprecate this field in favor of <code>RequeueAfter</code>

The `ctx` argument carries a logger with request fields and is cancelled when the manager shuts down. Pass it to every client call.

3.5 Level-triggered and edge-triggered logic

An edge-triggered controller acts on each change: "the replica count changed from 2 to 3, so create one Pod." If it misses an event, or two events are merged, its view of the world becomes wrong. A level-triggered controller compares the current desired state with the current actual state on every run: "three replicas are desired and two exist, so create one." It produces the correct result regardless of which events it received.

This is why a reconciler must be **idempotent**. It can run any number of times for the same object, and each run moves the actual state toward the latest desired state.

3.6 Rate limiting and backoff

The work queue uses a rate limiter that combines per-item exponential backoff with an overall limit. An object that fails repeatedly is retried after increasing delays, which prevents a persistent error from flooding the API server. When a reconcile succeeds, the queue forgets the item's failure history.

The metrics `controller_runtime_reconcile_total`, `controller_runtime_reconcile_errors_total`, and the `workqueue_*` family show this behavior in production.

4. Writing a reconciler

4.1 The structure of Reconcile

The following reconciler shows the standard sequence. Steps 1, 3, and 4 match the repository's controller; step 2 shows where deletion handling goes when an operator manages external resources. The repository's operator does not manage external resources and has no finalizer.

```
func (r *GpuWorkloadReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    logger := log.FromContext(ctx).WithValues("gpuworkload", req.NamespacedName)

    // 1. Fetch the latest object.
    workload := &gpucloudv1.GpuWorkload{}
    if err := r.Get(ctx, req.NamespacedName, workload); err != nil {
        if apierrors.IsNotFound(err) {
            // Object deleted: child objects are garbage collected via owner refs.
            return ctrl.Result{}, nil
        }
        return ctrl.Result{}, err // will requeue with backoff
    }

    // 2. Handle deletion (finalizer) if the API has marked it deleted.
    if !workload.DeletionTimestamp.IsZero() {
        return r.handleDeletion(ctx, workload)
    }

    // 3. Reconcile children: CreateOrUpdate Deployment and Service.
    if err := r.reconcileDeployment(ctx, workload); err != nil {
        return ctrl.Result{}, err
    }
    if err := r.reconcileService(ctx, workload); err != nil {
        return ctrl.Result{}, err
    }

    // 4. Read actual status from children and update the CR.
    if err := r.updateStatus(ctx, workload); err != nil {
        return ctrl.Result{}, err
    }

    return ctrl.Result{}, nil
}
```

The code follows four rules:

1. **Treat "not found" as success.** The object was deleted, and owner references remove its children, so there is nothing to do.
2. **Return other errors.** The queue retries them with backoff.
3. **Read the object before acting.** The request contains only a name.
4. **Use one helper per child resource,** so each step can be read and tested separately.

4.2 CreateOrUpdate

`controllerutil.CreateOrUpdate` implements the "make this object look like this" operation that most reconcilers need:

1. It reads the object from the cache.
2. If the object does not exist, it calls your *mutate* function and then `Create`.
3. If the object exists, it calls your mutate function on the existing object. If the function changed anything, it calls `Update`.
4. It returns whether it created, updated, or left the object unchanged.

```
_, err := controllerutil.CreateOrUpdate(ctx, r.Client, deployment, func() error {
    if err := controllerutil.SetControllerReference(workload, deployment, r.Scheme); err != nil {
        return err
    }

    // Fill the desired spec.
    deployment.Labels = labels
    deployment.Spec = appsv1.DeploymentSpec{ ... }

    return nil
})
```

Write the mutate function carefully:

- **Set only the fields you own.** If you replace `deployment.Spec` entirely, you erase fields that the API server filled with defaults. The object then differs from the stored version on every reconcile, and the controller sends an update each time.
- **Do not change immutable fields.** The API server rejects changes to fields such as a `Deployment`'s `spec.selector` or a `Service`'s `spec.clusterIP`.
- **Consider server-side apply** for objects that other controllers also modify. Server-side apply records which manager owns each field and changes only the fields your controller declares.

4.3 Objects from the cache

By default, `controller-runtime`'s cache returns a deep copy of the object on each `Get` and `List`, so modifying the returned object does not corrupt the cache. If you disable deep copies for performance (`UnsafeDisableDeepCopy`), you must call `DeepCopy()` before modifying an object. When you work with client-go informers directly, objects from the lister are shared with the cache and must never be modified.

4.4 Owner references and garbage collection

`controllerutil.SetControllerReference(owner, child, scheme)` adds an owner reference to the child:

```
ownerReferences:
- apiVersion: gpucloud.example.com/v1
  kind: GpuWorkload
  name: triton-demo
  uid: <uid>
  controller: true
  blockOwnerDeletion: true
```

The owner reference has two effects:

- **Garbage collection.** When the GpuWorkload is deleted, the Kubernetes garbage collector deletes the Deployment and Service.
- **Event mapping.** `Owns(&Deployment{})` uses the reference with `controller: true` to find which GpuWorkload to reconcile when the Deployment changes. An object can have only one controller owner.

Owner references work only within the cluster, and only within the same namespace for namespaced owners. Resources outside Kubernetes, such as cloud instances, DNS records, or quota reservations, need a finalizer.

4.5 Finalizers

A finalizer is a string in `metadata.finalizers` that prevents the API server from removing an object until a controller has finished cleaning up. Deletion then proceeds as follows:

1. A user deletes the object.
2. The API server sets `metadata.deletionTimestamp` and keeps the object, because the finalizer list is not empty.
3. The controller receives an update event, sees the deletion timestamp, and deletes the external resources.
4. When cleanup succeeds, the controller removes its finalizer from the list.
5. With no finalizers left, the API server removes the object.

```

const gpuInstanceFinalizer = "gpucloud.example.com/gpu-instance-cleanup"

func (r *GpuWorkloadReconciler) handleDeletion(ctx context.Context, workload *gpucloudv1.GpuWorkload)
(ctrl.Result, error) {
    if !controllerutil.ContainsFinalizer(workload, gpuInstanceFinalizer) {
        return ctrl.Result{}, nil
    }

    // Call the cloud provider API to terminate the GPU instance / release the
    // reservation. Retry until it succeeds.
    if err := r.terminateCloudInstance(ctx, workload); err != nil {
        return ctrl.Result{}, err
    }

    controllerutil.RemoveFinalizer(workload, gpuInstanceFinalizer)
    if err := r.Update(ctx, workload); err != nil {
        return ctrl.Result{}, err
    }
    return ctrl.Result{}, nil
}

```

Follow these rules:

- **Add the finalizer before creating the external resource.** If the controller creates the resource first and fails before adding the finalizer, a deletion can remove the object and leave the external resource running.
- **Make cleanup idempotent.** Treat "already deleted" as success, because cleanup can run more than once.
- **Expect stuck deletions if the controller is not running.** An object with a finalizer remains in Terminating until a controller removes the finalizer. This protects external resources from being leaked, but it also means that uninstalling the operator before deleting its resources leaves those resources undeletable until someone removes the finalizers manually.

Kubebuilder scaffolds an RBAC marker for the finalizers subresource. It is required to set `blockOwnerDeletion` on owner references in clusters that enable the `OwnerReferencesPermissionEnforcement` admission plugin:

```

// +kubebuilder:rbac:groups=gpucloud.example.com,resources=gpuworkloads/finalizers,verbs=update

```

4.6 Status, conditions, and `observedGeneration`

Report progress with standard conditions rather than individual boolean fields. The repository's status type is:

```

type GpuWorkloadStatus struct {
    ReadyReplicas    int32    `json:"readyReplicas,omitempty"`
    AvailableReplicas int32    `json:"availableReplicas,omitempty"`
    ObservedGeneration int64    `json:"observedGeneration,omitempty"`
    Phase            string   `json:"phase,omitempty"`
    Conditions       []metav1.Condition `json:"conditions,omitempty"`
}

```

Each `metav1.Condition` has these fields:

Field	Content
Type	A CamelCase name, such as Available, Progressing, or Ready
Status	True, False, or Unknown
Reason	A CamelCase machine-readable reason, such as DeploymentAvailable
Message	A human-readable explanation
ObservedGeneration	The object generation the condition was computed from
LastTransitionTime	When Status last changed

`LastTransitionTime` must change only when the condition's Status changes. Use `meta.SetStatusCondition` from `k8s.io/apimachinery/pkg/api/meta`, which preserves the existing time when the status is unchanged.

Note: The repository's `conditionsFor` function sets `LastTransitionTime` to the current time on every reconcile. `updateStatus` skips the write when the new status equals the old one, but the fresh timestamp means the two are almost never equal, so nearly every reconcile sends a status update, and each update that changes the stored object triggers another reconcile. Building the conditions with `meta.SetStatusCondition` on the existing list avoids the unnecessary writes.

`observedGeneration` lets clients tell whether the status reflects the latest spec:

```
kubectl get gpuworkload triton-demo -o jsonpath='{.status.observedGeneration}/{.metadata.generation}'
```

If `observedGeneration` is lower than `metadata.generation`, the controller has not yet processed the most recent spec change, and the status describes an older spec.

4.7 Update conflicts

Every object has a `resourceVersion`. An Update succeeds only if the object's `resourceVersion` still matches the stored one; otherwise the API server returns a 409

Conflict. This optimistic concurrency control prevents a controller from overwriting a change it has not seen. To handle it:

- Write status through the status subresource with `r.Status().Update` or `r.Status().Patch`.
- Read the latest object immediately before writing status. The repository's `updateStatus` helper does this.
- On a conflict, return the error. The queue retries, and the next reconcile reads the new version.
- Use a merge patch for small changes, so the request contains only the fields you change.

5. The Kubebuilder project

5.1 Layout

```

operator/
├── PROJECT                # Kubebuilder metadata: layout, group, version, kind
├── Makefile               # generate, manifests, test, and run targets
├── Dockerfile
├── go.mod
├── cmd/main.go            # manager entry point
├── api/v1/
│   ├── groupversion_info.go # GroupVersion and SchemeBuilder
│   ├── gpuworkload_types.go # spec and status types, with markers
│   └── zz_generated.deepcopy.go # generated DeepCopy methods
├── internal/controller/
│   ├── gpuworkload_controller.go # the reconciler and SetupWithManager
│   └── gpuworkload_controller_test.go
├── config/
│   ├── crd/bases/          # generated CRD
│   ├── rbac/              # generated ClusterRole
│   ├── manager/           # namespace for the operator
│   └── samples/            # example GpuWorkload resources
└── hack/                  # license header for generated files

```

A project created with `kubebuilder init` also contains `config/default/` and other Kustomize overlays for deploying the operator. This repository runs the manager locally, so it omits them.

5.2 The PROJECT file

PROJECT records how the project was scaffolded, so that later `kubebuilder create` commands generate consistent code. This repository uses the `go.kubebuilder.io/v4` layout, domain `example.com`, and one namespaced resource, `GpuWorkload`, in group `gpucloud` and version `v1`, with a controller.

5.3 Markers

Markers are comments that `controller-gen` reads. Type-level markers configure the CRD:

```

// +kubebuilder:object:root=true
// +kubebuilder:subresource:status
// +kubebuilder:resource:path=gpuworkloads,scope=Namespaced,shortName=gw
// +kubebuilder:printcolumn:name="Phase",type=string,JSONPath=`.status.phase`
// +kubebuilder:printcolumn:name="GPUs",type=integer,JSONPath=`.spec.gpuCount`
type GpuWorkload struct { ... }

```

Marker	Effect
<code>object:root=true</code>	Generate DeepCopyObject, so the type implements <code>runtime.Object</code>
<code>subresource:status</code>	Enable the <code>/status</code> subresource
<code>resource:path=...,scope=...,shortName=...</code>	Set the plural name, scope, and <code>kubectl</code> short name
<code>printcolumn</code>	Add columns to <code>kubectl get</code> output

Field-level markers add validation and defaults to the schema:

```
// +kubebuilder:validation:Required
// +kubebuilder:validation:Minimum=0
// +kubebuilder:default=1
// +optional
```

For rules that involve several fields, use a CEL validation rule, which the API server evaluates without a webhook:

```
// +kubebuilder:validation:XValidation:rule="self.gpuCount <= 8",message="gpuCount must be at most 8"
```

5.4 Code generation

```
make generate      # controller-gen object: DeepCopy methods
make manifests     # controller-gen rbac, crd, webhook: YAML under config/
make fmt vet       # gofmt and go vet
```

The CI workflow runs `make generate manifests` and then `git diff --exit-code`. The job fails if someone changed a Go type or marker without committing the regenerated files.

5.5 RBAC markers

The controller declares the permissions it needs with markers:

```
// +kubebuilder:rbac:groups=apps,resources=deployments,verbs=get;list;watch;create;update;patch;delete
// +kubebuilder:rbac:groups=core,resources=services,verbs=get;list;watch;create;update;patch;delete
// +kubebuilder:rbac:groups=core,resources=pods,verbs=get;list;watch
```

`make manifests` writes the resulting `ClusterRole` to `config/rbac/role.yaml`. When the operator is deployed, a `ClusterRoleBinding` grants that role to the manager's service account. The `list` and `watch` verbs are required for every type the controller watches, because the informer lists and watches it.

5.6 The manager entry point

`cmd/main.go` performs these steps:

1. Builds a Scheme and registers the built-in client-go types and the `gpucloud/v1` types.
2. Creates the manager with `ctrl.NewManager`, passing the metrics address, health probe address, and leader election settings.
3. Calls `SetupWithManager` for each reconciler.
4. Adds health and readiness checks.
5. Starts the manager with `ctrl.SetupSignalHandler()`, which cancels the context on `SIGTERM` or `SIGINT` so that the manager shuts down cleanly.

Common flags are `--metrics-bind-address`, `--health-probe-bind-address`, and `--leader-elect`.

5.7 Kubebuilder commands

```
kubebuilder init --domain example.com --repo github.com/you/gpu-cloud-operator
kubebuilder create api --group gpucloud --version v1 --kind GpuWorkload \
  --resource --controller
kubebuilder create webhook --group gpucloud --version v1 --kind GpuWorkload \
  --defaulting --programmatic-validation
```

- `init` creates the project skeleton, the manager, and the Makefile.
- `create api` adds the Go types and a controller scaffold.
- `create webhook` adds defaulting and validation webhook scaffolds and their configuration.

6. Admission webhooks

6.1 The admission sequence

When a client creates or updates an object, the API server processes the request in this order:

1. Authentication and authorization.
2. **Mutating admission**, including mutating webhooks, which can change the object.
3. Schema validation against the CRD schema.
4. **Validating admission**, including validating webhooks, which can reject the object but not change it.
5. Persistence to etcd.

A webhook is an HTTPS service that the API server calls. A `MutatingWebhookConfiguration` or `ValidatingWebhookConfiguration` tells the API server which operations and resources to send to it. Kubebuilder runs the webhook server inside the manager and generates the configuration. Because the API server calls the webhook synchronously, an unavailable webhook with `failurePolicy: Fail` blocks all writes to the resource.

6.2 Defaulting webhooks

Schema defaults (`+kubebuilder:default=1`) apply a fixed value when a field is omitted. A mutating webhook can compute defaults, for example setting a GPU toleration only when `gpuCount` is greater than zero, or choosing an image based on the GPU type.

6.3 Validating webhooks

A validating webhook enforces rules that the schema and CEL rules cannot express, such as checks against external data or policy:

- Reject a `gpuCount` above the tenant's allowed maximum.
- Reject images from registries that are not approved for the environment.
- Reject changes to a field that must not change after creation, such as the GPU type.

```
func (w *GpuWorkloadValidator) ValidateCreate(ctx context.Context, obj runtime.Object)
(admission.Warnings, error) {
    wl := obj.(*gpucloudv1.GpuWorkload)
    if wl.Spec.GpuCount > 8 {
        return nil, fmt.Errorf("gpuCount %d exceeds maximum 8", wl.Spec.GpuCount)
    }
    return nil, nil
}
```

Prefer schema validation and CEL rules where they are sufficient. They run inside the API server and add no availability risk.

7. Leader election, metrics, and health

7.1 Leader election

Running two or more replicas of the manager improves availability, but two replicas reconciling the same objects at the same time would conflict. With `--leader-elect`, the replicas compete for a Lease object in the operator's namespace. Only the replica that holds the lease runs the controllers; the others wait and take over if the leader stops renewing the lease. Webhook servers run on every replica, because they do not modify cluster state.

7.2 Metrics

controller-runtime serves Prometheus metrics at `/metrics`, including:

- Reconcile counts, errors, and duration for each controller
- Work queue depth, additions, retries, and the time items wait in the queue
- Requests from the client to the API server, by status code
- Go runtime metrics

Register application metrics with the controller-runtime metrics registry (`sigs.k8s.io/controller-runtime/pkg/metrics.Registry`), for example `gpu_workloads_ready` or `gpu_workload_time_to_ready_seconds`.

7.3 Health and readiness

The manager serves `/healthz` and `/readyz`:

```
mgr.AddHealthzCheck("healthz", healthz.Ping)
mgr.AddReadyzCheck("readyz", healthz.Ping)
```

The Deployment that runs the operator uses `/healthz` for its liveness probe and `/readyz` for its readiness probe. Keep the liveness check simple, so that a slow dependency does not cause Kubernetes to restart a working process. Readiness checks can include dependencies, such as the webhook server having loaded its certificates.

8. Testing an operator

08-testing-cicd-observability.md describes each test layer in detail. For an operator, the layers are:

Layer	Tool	What it verifies
Unit	Go tests on pure functions	Functions such as phaseFor, mergeLabels, and gpuResources
Unit	controller-runtime fake client	Reconcile creates the expected children, sets owner references and GPU resources, and updates status
Integration	envtest	The CRD schema, webhooks, manager startup, and RBAC
End-to-end	Kind	The full flow: install the CRD, run the operator, apply a sample, and wait for the Deployment to become available (scripts/verify-kind.sh)

Test pure functions with table-driven tests:

```
func TestPhaseFor(t *testing.T) {
    tests := []struct{ name string; desired, ready, available int32; want string }{
        {"zero desired", 0, 0, 0, ""},
        {"ready", 2, 2, 2, "Ready"},
        {"degraded", 3, 2, 2, "Degraded"},
        {"pending", 3, 0, 0, "Pending"},
    }
    for _, tc := range tests { ... }
}
```

9. Operator design checklist

Review an operator against each item:

- [] The spec is declarative and minimal, and the status reports observed state.
- [] Status uses conditions and observedGeneration.
- [] The status subresource is enabled.
- [] Reconcile reads the latest object and is idempotent.
- [] Child resources have controller owner references.
- [] External resources are protected by finalizers, added before the resource is created.
- [] Child resources are watched with owns.
- [] Objects are copied before modification when deep copies are disabled.
- [] Defaults and validation are declared with markers and CEL rules where possible.
- [] RBAC is generated from markers and grants only what the controller uses.
- [] The manager enables leader election, metrics, and health probes.
- [] Unit tests cover pure functions and reconcile logic with a fake client.
- [] envtest or Kind tests cover behavior against a real API server.
- [] Generated files are committed, and CI fails when they are out of date.
- [] Upgrades are compatible: new fields are optional, stored objects remain readable, and the manager rolls out without two leaders.

10. The GPU Operator as a case study

The NVIDIA GPU Operator applies these concepts at a larger scale:

- It is installed with Helm and configured through a cluster-scoped ClusterPolicy custom resource.
- It uses Node Feature Discovery to label nodes that have NVIDIA GPUs.
- It creates DaemonSets for each node-level component: the driver, the container toolkit, the device plugin, DCGM and its exporter, the MIG manager, and validators.
- It orders those components, because the device plugin cannot start until the driver and toolkit are ready.
- It must handle nodes that are added, rebooted, or upgraded, and components that fail on some nodes but not others.

`10-gpu-operator-kubebuilder-deep-dive.md` covers the GPU Operator in detail.

11. Interview questions

What is the difference between a controller and an operator?

A controller is a loop that reconciles actual state toward desired state for some resource type. An operator is a controller that manages a custom resource and encodes domain-specific operational knowledge, such as upgrades, backups, recovery, and cleanup. The Deployment controller is a controller; the GPU Operator is an operator.

Why are controllers level-triggered?

Events can be lost during a restart, merged in the work queue, or delivered out of order. A level-triggered reconciler reads the current desired and actual state on every run, so it reaches the correct result regardless of which events it received, and running it again is safe.

Why must `nvidia.com/gpu` be set in limits?

`nvidia.com/gpu` is an extended resource. Kubernetes requires extended resources to be specified in limits. If requests is also specified, it must equal limits, because extended resources cannot be overcommitted. A Pod that sets only requests fails validation.

What happens to a workload if the operator stops?

The Deployment and Service continue to run, because they are ordinary Kubernetes objects managed by built-in controllers. The custom resource's status stops updating, and changes to its spec are not applied until the operator runs again. A deleted resource with a finalizer stays in Terminating until the operator removes the finalizer.

How do you upgrade an operator safely?

Make API changes additive: add optional fields rather than renaming or removing fields, and provide conversion when a new version changes the schema. Keep the storage version readable. Use leader election so that old and new replicas do not reconcile at the same time during the rollout. Test the new CRD against existing objects with `envtest`, and keep finalizer and cleanup behavior compatible.

What metrics would you monitor for an operator?

Reconcile rate, errors, and duration; work queue depth and retries; failures creating child objects; the number of custom resources in each phase, such as Ready, Pending, and Failed; and time from creation to Ready.

Summary

- An operator is a CRD plus a controller that encodes operational knowledge and runs continuously.
- controller-runtime watches resources through informers, deduplicates work in a rate-limited queue, and calls `Reconcile` with only a name.
- Reconcilers are level-triggered and idempotent: read the latest object, converge the children, and report status.
- Owner references clean up in-cluster children; finalizers clean up external resources.
- Use conditions with `observedGeneration`, and change `LastTransitionTime` only when a condition's status changes.
- Kubebuilder markers generate the CRD schema, RBAC, and deep-copy code, and CI should verify that generated files are current.

Source: [04-kubernetes-operator.md](#) · [PDF](#) · [All study guides](#)